

Guida rapida al tuo progetto: Emulatore CHIP-8

Questo file è una piccola guida per il tuo progetto personale: scrivere un **emulatore CHIP-8** per imparare concetti di informatica di base (CPU, memoria, grafica semplice, input) divertendoti con qualche giochino.

Niente codice, solo concetti, roadmap e link utili.

1. Che cos'è un emulatore (in breve)

Un **emulatore** è un programma che fa finta di essere un altro computer o una console.

- Il tuo PC reale è il **sistema host** (Windows, Linux, ecc.).
- La macchina che vuoi simulare (in questo caso CHIP-8) è il **sistema guest**.
- L'emulatore:
 - mantiene una **memoria finta** (un array di byte);
 - mantiene dei **registri finti** (il "cervello" della CPU virtuale);
 - legge le **istruzioni** di un programma compilato per quella macchina;
 - le esegue una alla volta (**fetch** → **decode** → **execute**);
 - aggiorna **schermo**, **input**, **timer** come farebbe l'hardware reale.

Risultato: puoi far girare programmi pensati per quella macchina "virtuale" sulla tua macchina reale.

2. Che cos'è CHIP-8

CHIP-8 non è un chip fisico: è una **macchina virtuale** molto semplice, inventata negli anni '70 per creare piccoli giochi su microcomputer dell'epoca.

A livello concettuale, CHIP-8 ha:

- **Memoria**: 4 KB totali.
- **Registri generali**: 16 registri a 8 bit, di solito chiamati `V0 ... VF` (nei documenti).
- **Registro indice**: un registro speciale per puntare nella memoria (spesso chiamato `I`).
- **Program counter**: indica l'indirizzo dell'istruzione corrente.
- **Stack** + stack pointer: per gestire le chiamate a sottoprogrammi (call/return).
- **Display**:
 - risoluzione 64x32 pixel;
 - bianco/nero (o acceso/spento).
- **Input**: 16 tasti (0–F), che tu mapperai alla tastiera del PC.
- **Timer**: due timer che scalano a 60 Hz (uno "delay", uno "sound").

Le **istruzioni** CHIP-8 (opcode) sono poche, qualche decina, e ognuna occupa 2 byte. Implementarle significa decidere, per ogni opcode:

“Quando vedo questa istruzione, cosa devo fare a memoria, registri, schermo, timer, input?”

In pratica, stai costruendo un **piccolo computer finto**, su cui poi far girare giochi già scritti in passato.

3. Risorse fondamentali su CHIP-8 ed emulatori

Qui trovi un recap delle risorse citate in chat, organizzate per tipo.

3.1 Documentazione e teoria su CHIP-8

- **Wikipedia – CHIP-8**
Panoramica generale su cos'è CHIP-8, un po' di storia e una descrizione dei componenti principali (memoria, registri, display, tastiera, timer). Utile come primo sguardo d'insieme.
- **Cowgod's CHIP-8 Technical Reference**
Una delle reference classiche. Spiega in dettaglio:
 - layout della memoria;
 - registri;
 - mappatura del display;
 - mappatura dei tasti;
 - descrizione *una per una* di tutte le istruzioni (opcode).
- **Articolo "Building a CHIP-8 Emulator [C++]" (Austin Morlan)**
Tutorial passo-passo in C++ che costruisce un emulatore CHIP-8. Anche se non vuoi copiare il codice, è utilissimo per capire:
 - come spezzare il progetto in parti;
 - quali strutture dati ti servono;
 - come collegare la parte "core" (CPU virtuale) con grafica e input.
- **Articolo/serie "Let's Have Fun With Interpreters and Bytecode VMs – CHIP-8"**
Un'altra spiegazione discorsiva di come trasformare il bytecode CHIP-8 in un interprete/emulatore:
 - concetto di bytecode;

- ciclo di interpretazione;
- design della VM.

Usa queste risorse per chiarirti *il modello mentale* di cosa è CHIP-8 e quali pezzi deve avere il tuo emulatore.

3.2 Raccolte di ROM (giochi) per CHIP-8

Queste sono le ROM (giochi/programmi) che potrai far girare sul tuo emulatore.

Nota: in genere queste raccolte sono in **public domain** o con licenza che ne permette l'uso e la distribuzione. Controlla comunque sempre le note sui siti.

1. Chip-8 Games Pack (Zophar's Domain / archivi simili)

Pacchetto "classico" con molti giochi storici, spesso in public domain. Dentro troverai (nomi indicativi, possono cambiare leggermente):

- Pong / Pong 2
- Tetris
- Invaders (tipo Space Invaders)
- Brix / Vbrix (breakout-style)
- 15 Puzzle
- TicTac (tris)
- Tank, UFO
- altri piccoli puzzle e demo

È un ottimo pacchetto base: pochi KB totali, ma un bel po' di gioco per test.

2. Public domain ROM pack (Internet Archive / mirror vari)

Un altro archivio di ROM CHIP-8 indicate come public domain, spesso simile o sovrapposto al pacchetto precedente, ma utile come backup.

3. GitHub – kripod/chip8-roms

Grande raccolta moderna di ROM, organizzata per cartelle:

- `chip8/` → giochi e demo classici;
- `schip/` → ROM per SuperCHIP (estensione di CHIP-8);
- `test/` → ROM di test.

Ti basta clonare o scaricare il repository e puntare il tuo emulatore alle ROM nella sottocartella dei giochi.

4. CHIP-8 Archive (John Earnest)

Archivio moderno con programmi CHIP-8 (spesso creati per jam come Octojam), con licenze esplicite (spesso CC0):

- molti giochi originali moderni;
- diverse demo;
- a volte anche cose sperimentali o artistiche.

È perfetto per testare il tuo emulatore su ROM un po' più "creative".

5. ROM di test – Timendus/chip8-test-suite

Non sono giochi, ma:

- verificano se implementi correttamente le istruzioni;
- controllano grafica, input, timer e altri dettagli.

Sono molto utili nella fase finale, quando i giochi "quasi funzionano" ma hai qualche bug strano.

4. Roadmap di sviluppo (senza codice)

Questa è una proposta di percorso per lavorare al progetto circa **1–2 ore al giorno per un mese**. Prendila come guida flessibile, non come regola rigida.

Settimana 1 – Capire il problema e impostare l'ambiente

Obiettivi:

- Capire *cos'è* CHIP-8 (non a memoria, ma concettualmente).
- Preparare un ambiente di sviluppo comodo sul tuo sistema (per esempio Windows).

Cose da fare:

1. Leggere:
 - Wikipedia CHIP-8;
 - a grandi linee Cowgod's reference (almeno sezioni memoria/registri/display).
2. Prendere appunti sui componenti della VM:
 - cosa contiene la memoria;
 - quali registri ti servono;
 - come funziona il display 64×32;
 - come funzionano i timer.
3. Scegliere:
 - linguaggio principale (C o C++);
 - libreria per grafica/input (SDL2 è una scelta semplice e portabile).
4. Verificare di riuscire ad aprire **una finestra vuota** sul desktop (ma senza ancora collegarla a CHIP-8).

Alla fine di questa settimana dovresti:

- avere un'idea mentale chiara di "com'è fatto un CHIP-8";
 - avere un progetto vuoto che si compila, con una semplice finestra che si apre e si chiude.
-

Settimana 2 – Modellare la macchina virtuale CHIP-8

Obiettivi:

- Disegnare la struttura logica della tua VM.
- Capire il ciclo "fetch → decode → execute".

Cose da fare (solo concetto, non codice qui nel file):

1. Decidere come rappresentare:
 - la memoria (un blocco contiguo di byte);
 - i 16 registri generali;
 - il registro indice;
 - il program counter;
 - lo stack (per le chiamate a sottoprogrammi);
 - i timer;
 - lo stato dei tasti;
 - il framebuffer 64×32 (array di pixel accesi/spenti).
2. Definire un ciclo principale a grandi linee:
 - leggere l'istruzione corrente in memoria;
 - avanzare il program counter;
 - riconoscere quale opcode è;
 - eseguire l'azione corrispondente;
 - aggiornare timer e schermo con una certa frequenza.
3. Leggere dagli articoli/tutorial:
 - come altri autori organizzano la VM;
 - quali gruppi di istruzioni implementano per primi (jump, registri, clear screen, disegno sprite, ecc.).

Alla fine di questa settimana dovresti:

- avere su carta (o in un documento) la "mappa" della tua VM;
 - sapere quali campi deve contenere il tuo oggetto "Chip-8";
 - avere un'idea di quali istruzioni affronterai per prime.
-

Settimana 3 – Ciclo di esecuzione, grafica e input

Obiettivi:

- Far sì che la tua VM legga ed esegua davvero istruzioni.
- Vedere qualcosa sullo schermo.

Cose da fare (a livello di idea):

1. Implementare il ciclo di esecuzione:
 - per ogni passo:
 - recuperare l'istruzione dalla memoria;
 - interpretarla;
 - modificare memoria/registri/pc/stack/graphics a seconda del caso.
2. Integrare il viewport grafico:
 - collegare il framebuffer 64×32 alla finestra;
 - decidere quanto ingrandire ogni pixel (es. 10×) per riempire la finestra.
3. Collegare l'input:
 - mappare 16 tasti "virtuali" a tasti reali della tastiera (per esempio 1-4, Q-R, A-F, Z-V o simili).
4. Gestire il tempo:
 - decidere quante istruzioni per secondo eseguire;
 - aggiornare i timer CHIP-8 circa a 60 Hz.

Alla fine di questa settimana l'ideale è riuscire a vedere **almeno un ROM semplice** disegnare qualcosa sullo schermo (anche se non funziona tutto alla perfezione).

Settimana 4 – Test, giochi e rifiniture

Obiettivi:

- Rendere il tuo emulatore abbastanza stabile da far girare 2-3 giochi senza glitch evidenti.
- Migliorare un po' l'esperienza d'uso.

Cose da fare:

1. Provare giochi semplici:
 - Pong / Pong 2;
 - Tetris;
 - Invaders;
 - Brix/Vbrix;
 - altri piccoli giochi dal pacchetto ROM.
2. Quando qualcosa non funziona:
 - controllare l'opcode specifico coinvolto;
 - confrontare il comportamento atteso con la reference (Cowgod, test ROM).

3. Usare le ROM di test (`chip8-test-suite`) per:
 - verificare la correttezza di:
 - istruzioni di salto, call/return;
 - operazioni sui registri;
 - disegno sprite e collisioni;
 - gestione dei timer;
 - input.
4. Piccole migliorie possibili:
 - supporto a pausa / reset;
 - un piccolo menù o opzione da linea di comando per scegliere la ROM;
 - qualche statistica (frame al secondo, istruzioni al secondo).

Alla fine di questa settimana il tuo obiettivo è: **poter aprire il programma, selezionare una ROM CHIP-8 e giocare** con un risultato soddisfacente.

5. Suggerimenti di lavoro e mindset

- **Non puntare alla perfezione subito.**
Inizia con pochi opcode fondamentali e con un ROM molto semplice. Meglio un emulatore incompleto che però fa qualcosa, piuttosto che niente.
- **Usa i giochi come “test automatici”.**
Se Pong mostra la pallina che si muove male o i paddle si comportano in modo strano, probabilmente hai sbagliato una delle istruzioni di salto o le operazioni sui registri.
- **Tieni un diario di bordo.**
Annota ogni giorno:
 - cosa hai capito meglio;
 - cosa ancora non è chiaro;
 - eventuali bug da indagare il giorno dopo.
- **Ricordati che l'obiettivo è imparare e divertirti.**
Non serve che alla fine il tuo emulatore sia il più veloce o completo del mondo: il valore sta nei concetti che avrai capito (VM, ciclo di esecuzione, interpretazione di bytecode, grafica semplice, input, timing).

Quando ti sentirai a tuo agio con CHIP-8, questo progetto sarà un ottimo trampolino per passare a cose ancora più “hardcore” (altri emulatori, compilatori semplici, interpreti di linguaggi, sistemi operativi didattici, ecc.).

Buon divertimento con il tuo emulatore CHIP-8! 🎮

If your Markdown is in a file clear this content and drop your file into this editor.

tip: click on the pencil icon on the left to clear the editor)

GitHub flavoured styling by default

We now use GitHub flavoured styling by default.